

Storage Management

Main Memory Background

Memory is central to the operation of computing systems. It's essentially a large array of words or bytes, each with its own address. Memory contains both the program to be executed and the data it operates on. The CPU fetches instructions from memory based on the **program counter**, which holds the address of the next instruction to be executed.

A typical instruction-execution cycle involves:

1. Fetching an instruction from memory.
2. Decoding the instruction and fetching operands from memory, if necessary.
3. Executing the instruction on the operands.
4. Storing the results back in memory.

The memory unit sees a stream of addresses along with read or write requests.

Basic Hardware and Memory Access

The CPU can directly access main memory and processor registers. Instructions use main memory addresses as program arguments, not disk addresses. Therefore, data and the instructions that execute them must reside in main memory.

Key issues and concerns include:

- **Speed of access:** Addressed by using **cache memory**.
- **Protecting program data:** Achieved through **base and limit registers**.
- **Size of memory:** Managed using **swapping** and **virtual memory**.

Registers built into the CPU are generally accessible within one CPU clock cycle or less. Main memory access can take many cycles, potentially causing the CPU to stall.

Speed of Access and Cache Memory

One of the critical parameters for judging speed is the number of CPU clock cycles needed to perform a memory operation. CPUs are generally faster than memory. CPU registers are accessible in one CPU clock cycle, but main memory access requires a transaction on the memory bus, which can take many CPU clocks to complete. This can lead to CPU stalling because the CPU has the program but lacks the data to complete instruction execution.

The solution to memory stalls is to introduce a faster memory between the CPU and main memory, known as **cache memory**. Cache memory acts as a buffer to accommodate the speed difference between the CPU and main memory.

Memory Protection

Aside from the relative speed of accessing physical memory, it's crucial to ensure correct operation by protecting:

- The operating system from unauthorized access by user processes.
- User processes from each other.

This protection is typically provided by the hardware. Each process has a range of legal addresses it can access, enforced by **base** and **limit registers**.

- **Base Register:** Holds the smallest legal physical memory address.
- **Limit Register:** Specifies the size of the range.

For example, if the base register holds **300040** and the limit register is **120900**, the program can legally access addresses from **300040** through **420939** (inclusive).

The CPU hardware compares every address generated in user mode with these registers. Any attempt to access operating system memory or other users' memory results in a trap to the operating system, which treats it as a fatal error. This prevents user programs from accidentally or deliberately modifying the code or data structures of the operating system or other users.

Only the operating system, executing in kernel mode, can load the base and limit registers using special privileged instructions. This prevents user programs from changing the registers' contents, while giving the OS unrestricted access to both OS and user memory.

Address Binding

Typically, a program resides on disk as a binary executable file. To be executed, it must be brought into memory and placed within a process. Programs on disk, ready to be loaded into memory for execution, form an **input queue**.

As a process executes, it accesses instructions and data from memory. Eventually, the process terminates, and its memory space is freed. User processes can reside in any part of main memory, and the user process doesn't necessarily need to be loaded at address **00000**.

Addresses in a source program are symbolic. The compiler binds these symbolic addresses to **relocatable addresses**. The linker or loader then turns relocatable addresses into **absolute addresses**. Each such binding is a mapping from one address space to another.

Addresses are represented in various ways during a program's lifecycle:

- **Symbolic Addresses**: Used in the source code (e.g., variable names).
- **Relocatable Addresses**: Assigned by the compiler (e.g., "14 bytes from the beginning of this module").
- **Absolute Addresses**: Assigned by the linker or loader (e.g., **74014**).

Binding of Instructions and Data to Memory

Address binding of instructions and data to memory addresses can happen at three different stages:

- Compile time
- Load time
- Execution time

The timing depends on when the address information becomes available.

Binding Time	Description	Implications
Compile Time	If the memory location is known at compile time, absolute code is generated.	Requires recompilation if the starting location changes.
Load Time	If the memory location is known at load time but not at compile time, relocatable code is generated.	Final binding is delayed until load time. Only requires reloading the code if the starting address changes.
Execution Time	If the process can be moved from one memory segment to another during execution, binding is delayed until run time.	Requires special hardware support.

Logical vs. Physical Address Space

- **Logical Address:** An address generated by the CPU, also referred to as a virtual address.
- **Physical Address:** The address loaded into the memory address register of the memory unit.

The **logical address space** is the set of all addresses generated by a program, while the **physical address space** is the set of all physical addresses corresponding to those logical addresses.

In compile-time and load-time address-binding schemes, logical and physical addresses are the same. However, in the execution-time address-binding scheme, they differ.

Memory-Management Unit (MMU)

The run-time mapping from virtual to physical addresses is done by a hardware device called the MMU.

Many methods are possible for this mapping. A simple MMU scheme generalizes the base-register scheme, where the base register is called the **relocation register**. The value in the relocation register is added to every address generated by a user process at the time it is sent to memory.

For example, if the base is at 14000, an attempt to address location 0 is dynamically relocated to location 14000, and an access to location 346 is mapped to location 14346. The user program only sees logical addresses, while the memory mapping hardware converts these to physical addresses. Logical addresses range from 0 to max, and physical addresses range from $R+0$ to $R+max$, where R is the base value.

Dynamic Loading

Normally, the entire program and all data of a process must be in physical memory for the process to execute, limiting the size of a process to the size of physical memory. **Dynamic loading** is used to obtain better memory-space utilization.

With dynamic loading, a routine is not loaded until it is called. All routines are kept on disk in relocatable load format.

1. The main program is loaded into memory and executed.
2. When a routine needs to call another routine, the calling routine first checks if the other routine has been loaded.
3. If not, the relocatable linking loader is called to load the desired routine into memory and update the program's address tables.
4. Control is then passed to the newly loaded routine.

The advantage of dynamic loading is that unused routines are never loaded, which is useful when large amounts of code are needed for infrequently occurring cases (e.g., error routines). This is implemented through program design, and operating systems may provide library routines to support dynamic loading.

Dynamic Linking

- **Static Linking:** System libraries and program code are combined by the linker/loader into the binary program image.
- **Dynamic Linking:** Linking is postponed until execution time.

Dynamic linking is commonly used with system libraries. Without dynamic linking, each program must include a copy of its language library in the executable image, wasting both disk space and main memory.

A **stub** is included in the image for each library routine reference.

The stub is a small piece of code that indicates how to locate the appropriate memory-resident library routine or how to load the library if the routine is not already present.

When the stub is executed:

1. It checks to see whether the needed routine is already in memory.
2. If not, the program loads the routine into memory.
3. The stub replaces itself with the address of the routine and executes the routine.
4. The next time that code segment is reached, the library routine is executed directly, without incurring the cost of dynamic linking.

Dynamic linking also facilitates library updates. A library can be replaced by a new version, and all programs that reference the library will automatically use the new version without needing to be relinked.

Swapping

Swapping is a process of temporarily swapping a process to a secondary memory from the main memory.

A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution. The **backing store** is a fast disk large enough to accommodate copies of all memory images for all users and must provide direct access to these memory images.

In a multiprogramming environment with a round-robin CPU-scheduling algorithm:

1. When a quantum expires, the memory manager swaps out the process that just finished and swaps another process into the freed memory space.
2. The CPU scheduler allocates a time slice to another process in memory.
3. This process repeats as each process finishes its quantum.

Ideally, the memory manager swaps processes fast enough so that processes are in memory, ready to execute, when the CPU scheduler wants to reschedule the CPU.

A variant of swapping used for priority-based scheduling algorithms is called **roll out, roll in**:

1. If a higher-priority process arrives and wants service, the memory manager swaps out the lower-priority process.
2. The higher-priority process is loaded and executed.
3. When the higher-priority process finishes, the lower-priority process is swapped back in and continued.

The major component of swap time is transfer time, which is directly proportional to the amount of memory swapped. The system maintains a ready queue of ready-to-run processes with memory images on disk.

Typically, a swapped-out process is swapped back into the same memory space it occupied previously. This restriction depends on the method of address binding. If binding is done at compile or load time, the process cannot be easily moved. If execution-time binding is used, a process can be swapped into a different memory space.

The dispatcher checks to see whether the next process in the queue is in memory. If it is not, the swapper swaps out a process currently in memory and swaps in the desired process. Then, it reloads registers and transfers control to the selected process.

The context-switch time in a swapping system can be significant. For example, swapping a 10 MB process to or from main memory using a hard disk with a transfer rate of 40 MB per second, with an average latency of 8 milliseconds, results in a swap time of 258 milliseconds. Considering both swap out and swap in, the total swap time is 516 milliseconds. For efficient CPU utilization, the execution time for each process should be long relative to the swap time.

Modified versions of swapping are found on many systems, such as UNIX, Linux, and Windows. Swapping is normally disabled and started only if memory allocation exceeds a threshold. It is disabled again once memory demand falls below the threshold.

Contiguous Allocation

Main memory must support both the operating system and user processes efficiently. Contiguous allocation, where each process is contained in a single contiguous section of memory, is one early method of memory allocation.

The memory is divided into two partitions:

- One for the resident operating system.
- One for the user processes.

The operating system can be located in either low or high memory. The location of the interrupt vector often dictates this decision, with the operating system commonly placed in low memory. User processes are then held in high memory.

To protect user processes from each other and from modifying operating-system code and data, a **relocation register** and a **limit register** are used.

1. The relocation register contains the value of the smallest physical address.
2. The limit register contains the range of logical addresses.

Each logical address must be less than the limit register, and the MMU maps the logical address dynamically by adding the value in the relocation register. This mapped address is sent to memory.

Types of Memory Management Techniques

Memory management techniques can be broadly classified into two categories:

- **Contiguous Allocation:** The executing process must be loaded entirely in the main memory in a single, contiguous block.
 - **Fixed (or Static) Partitioning:** The number of partitions in RAM is fixed, although the size of each partition may vary.
 - **Variable (or Dynamic) Partitioning:** Partitions are created during run-time based on the needs of the processes.
- **Non-Contiguous Allocation:** A process is allocated a series of non-contiguous blocks of memory that can be located anywhere in the physical memory.

Fixed Partitioning

Also known as static partitioning, the number of partitions are fixed.

Fixed partitioning involves dividing the physical memory into fixed-size partitions or regions before execution or during system configuration. Each partition is dedicated to a specific process until it terminates or releases the partition.

A disadvantage of fixed partitioning is that it can lead to **internal fragmentation**, where memory within a partition remains unused because the process's memory requirements are smaller than the partition size.

Variable (or Dynamic) Partitioning

In contrast to fixed partitioning, partitions are not created before execution or during system configuration. Instead, partitions are made during run-time according to the process's needs.

Initially, RAM is empty, and partitions are created as processes arrive. The size of the partition is equal to the incoming process's needs, which avoids internal fragmentation and ensures efficient utilization of RAM. The number of partitions in RAM is not fixed and depends on the number of incoming processes and the size of the main memory.

Non-Contiguous Memory Allocation

In this technique, each process is allocated a series of non-contiguous blocks of memory that can be located anywhere in the physical memory. **Paging** and **segmentation** are two methods that allow a process's physical address space to be non-contiguous.

Non-contiguous memory allocation reduces memory wastage but increases overhead due to address translation, slowing down execution.

Memory Allocation - Fixed Size Partitions

One of the simplest methods for allocating memory is to divide it into several fixed-sized partitions, each of which can contain exactly one process. The degree of multiprogramming is limited by the number of partitions. When a partition is free, a process is selected from the input queue and loaded into the free partition. When the process terminates, the partition becomes available for another process. This method was originally used by the IBM OS/360 operating system (called MFT) but is no longer in use.

Memory Allocation - Variable Partition Scheme

In the variable partition scheme, the operating system maintains a table indicating which parts of memory are available and which are occupied. Initially, all memory is available for user processes and is considered one large block of available memory—a **hole**. Over time, memory contains a set of holes of various sizes.

Hole: Block of available memory; holes of various size are scattered throughout memory

As processes enter the system, they are placed into an input queue. The operating system can order the input queue according to a scheduling algorithm. A process selected from the input queue is allocated memory from a hole large enough to accommodate it. The operating system can wait until a large enough block is available or skip down the input queue to see whether the smaller memory requirements of some other process can be met.

When a process terminates, it releases its block of memory, which is then placed back in the set of holes. If the new hole is adjacent to other holes, these adjacent holes are merged to form one larger hole. At this point, the system may need to check whether there are processes waiting for memory and whether this newly freed and re combined memory could satisfy the demands of any of these waiting processes.

Dynamic Storage-Allocation Problem

The dynamic storage-allocation problem involves satisfying a request of size **n** from a list of free holes. Several strategies exist:

- **First-Fit:** Allocate the first hole that is big enough.
- **Best-Fit:** Allocate the smallest hole that is big enough.
- **Worst-Fit:** Allocate the largest hole.

Algorithm	Description	Pros	Cons
First-Fit	Allocate the first hole that is big enough.	Simple and fast.	May result in larger holes being allocated, leading to fragmentation.
Best-Fit	Allocate the smallest hole that is big enough.	Minimizes the size of leftover holes.	Requires searching the entire list of holes, which can be slow.
Worst-Fit	Allocate the largest hole.	May leave larger holes for future allocations.	Requires searching the entire list and may lead to smaller, less useful holes over time.

Simulations have shown that both first-fit and best-fit perform better than worst-fit in terms of decreasing time and storage utilization. First-fit is generally faster than best-fit, but neither is clearly better in terms of storage utilization.

Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into smaller pieces, leading to fragmentation. There are two main types of fragmentation:

- External Fragmentation
- Internal Fragmentation

External Fragmentation

Total memory space exists to satisfy a request, but it is not contiguous.

Storage is fragmented into a large number of small holes. In the worst case, there is a block of free (or wasted) memory between every two processes.

Internal Fragmentation

Allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition but not being used.

This is unused memory internal to a partition.

For example, consider a multiple-partition allocation scheme with a hole of 18,464 bytes. If the next process requests 18,462 bytes and we allocate exactly the requested block, we are left with a hole of 2 bytes, which may be unusable.

First-fit analysis reveals that given N blocks allocated, approximately $0.5N$ blocks are lost to fragmentation, with about $1/3$ being unusable, following the 50-percent rule.

Compaction

External fragmentation can be reduced by **compaction**, which involves shuffling memory contents to place all free memory together in one large block. Compaction is possible only if relocation is dynamic and done at execution time.

If addresses are relocated dynamically, relocation requires only moving the program and data and then changing the base register to reflect the new base address. The simplest compaction algorithm moves all processes toward one end of memory, with all holes moving in the other direction, producing one large hole of available memory. This scheme can be expensive.

Non-Contiguous Memory Allocation

Another solution to fragmentation is to permit the logical address space of processes to be non-contiguous, allowing a process to be allocated physical memory wherever such memory is available. Paging and segmentation are two ways to achieve this, reducing memory wastage but increasing overhead due to address translation.

Paging

Paging is a memory-management scheme that permits the physical address space of a process to be non-contiguous. This means that a process is allocated physical memory whenever it is available, avoiding external fragmentation and the need for compaction. It also avoids the problem of varying sized memory chunks.

Memory Division

Paging involves dividing both physical and logical memory:

- **Physical memory** is divided into fixed-size blocks called **frames**.
 - The size is a power of 2, typically between 512 bytes and 16 MB.
- **Logical memory** is divided into blocks of the same size called **pages**.

To run a program of size N pages, the system needs to find N free frames and load the program into them. A **page table** is then set up to translate logical to physical addresses. The backing store is also split into pages.

Address Translation Scheme

The address generated by the CPU is divided into:

- **Page number (p)**: Used as an index into a page table. The page table contains the base address of each page in physical memory.
- **Page offset (d)**: Combined with the base address to define the physical memory address that is sent to the memory unit.

Paging Hardware

Paging Model of Logical and Physical Memory

The **page size**, like the **frame size**, is defined by the hardware and is typically a power of 2, varying between 512 bytes and 16 MB per page, depending on the computer architecture. Choosing a power of 2 as a page size simplifies the translation of a logical address into a page number and page offset.

If the size of the logical address space is 2^m , and a page size is 2^n addressing units (bytes or words), then:

- The high-order $m - n$ bits of a logical address designate the page number.
- The n low-order bits designate the page offset.

Misplaced \hline

Where:

- **p** is an index into the page table
- **d** is the displacement within the page

Paging Example

Given a logical address space of 2^m and a page size of 2^n , consider an example where $n = 2$ and $m = 4$. This means the logical address space is 2^4 (16 bytes), and the page size is 2^2 (4 bytes). Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages), the user's view of memory can be mapped into physical memory.

Misplaced \hline

For instance, with a 32-byte memory and 4-byte pages:

1. **Logical address 0** (page 0, offset 0) maps to physical address 20 because page 0 is in frame 5. The calculation is $(5 \times 4) + 0 = 20$.
2. **Logical address 3** (page 0, offset 3) maps to physical address 23 because page 0 is in frame 5. The calculation is $(5 \times 4) + 3 = 23$.
3. **Logical address 4** (page 1, offset 0) maps to physical address 24 because page 1 is mapped to frame 6. The calculation is $(6 \times 4) + 0 = 24$.
4. **Logical address 13** (page 3, offset 1) maps to physical address 9 because page 3 is mapped to frame 2. The calculation is $(2 \times 4) + 1 = 9$.

Fragmentation

With paging, there is no external fragmentation because any free frame can be allocated to a process that needs it. However, there may be some internal fragmentation.

Calculating Internal Fragmentation

Consider the following:

- Page size = 2,048 bytes
- Process size = 72,766 bytes (35 pages + 1,086 bytes)

Internal fragmentation = $2,048 - 1,086 = 962$ bytes

In the worst-case scenario, a process might need n pages plus 1 byte. It would be allocated $n + 1$ frames, resulting in internal fragmentation of almost an entire frame.

Free Frames

When a process arrives in the system to be executed, its size, expressed in pages, is examined. Each page of the process needs one frame. If the process requires n pages, at least n frames must be available in memory. If n frames are available, they are allocated to this arriving process. The first page of the process is loaded into one of the allocated frames, and the frame number is put in the page table for this process. This continues for each page.

User's View vs. Physical Memory

Paging provides a clear separation between the user's view of memory and the actual physical memory. The user program views memory as a single, contiguous space containing only this one program. In reality, the user program is scattered throughout physical memory, which also holds other programs.

Address Translation

The difference between the user's view of memory and the actual physical memory is resolved by the address-translation hardware. The logical addresses are translated into physical addresses. This mapping is hidden from the user and is controlled by the operating system.

Hardware Support

Each operating system has its own methods for storing page tables. Most allocate a page table for each process. A pointer to the page table is stored with the other register values (like the instruction counter) in the process control block (PCB). When the dispatcher is told to start a process, it must reload the user registers and define the correct hardware page-table values from the stored user page table.

Implementation of Page Table

The hardware implementation of the page table can be done in several ways:

1. Using Registers:

- The page table is implemented as a set of dedicated registers. These registers should be built with very high-speed logic to make the paging-address translation efficient.
- Suitable if the page table is reasonably small (e.g., 256 entries).
- Not feasible for large page tables (e.g., 1 million entries) due to the cost.

2. Using Main Memory:

- The page table is kept in main memory.
- **Page-table base register (PTBR)** points to the page table.
- **Page-table length register (PTLR)** indicates the size of the page table.
- Changing page tables requires changing only the PTBR.
- Every data/instruction access requires two memory accesses: one for the page table and one for the data/instruction.

The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers (TLBs)**.

Translation Look-Aside Buffers (TLBs)

A **TLB** is a special fast-lookup hardware cache. Each entry in the TLB consists of two parts:

- A key (or tag)
- A value

When the associative memory is presented with an item, the item is compared with all keys simultaneously. If the item is found, the corresponding value is returned. The search is fast but expensive. Typically, the number of entries in a TLB is small, often between 64 and 1,024.

How TLBs Work

1. The TLB contains only a few of the page-table entries.
2. When a logical address is generated by the CPU, its page number is presented to the TLB.
3. If the page number is found (TLB hit), its frame number is immediately available and is used to access memory.
4. If the page number is not in the TLB (TLB miss):
 - A reference to the page table must be made.
 - When the frame number is obtained, it is used to access memory.
 - The page number and frame number are added to the TLB for future references.
5. If the TLB is full, the operating system must select one entry for replacement, using policies like least recently used (LRU) or random replacement.

Associative Memory

Associative memory performs a parallel search.

Misplaced \hline

For address translation (p, d):

- If p is in the associative register, get frame # out.
- Otherwise, get frame # from the page table in memory.

Address-Space Identifiers (ASIDs)

Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry. An ASID uniquely identifies each process and provides address-space protection for that process.

1. When the TLB attempts to resolve virtual page numbers, it ensures that the ASID for the currently running process matches the ASID associated with the virtual page.
2. If the ASIDs do not match, the attempt is treated as a TLB miss.

An ASID allows the TLB to contain entries for several different processes simultaneously, in addition to providing address-space protection.

Effective Access Time

Hit Ratio: The percentage of times that a particular page number is found in the TLB.

For example, an 80-percent hit ratio means that the desired page number is found in the TLB 80 percent of the time.

- **TLB Hit:**
 - It takes 20 nanoseconds to search the TLB.
 - It takes 100 nanoseconds to access memory.
 - A mapped-memory access takes 120 nanoseconds.
- **TLB Miss:**
 - It takes 20 nanoseconds to search the TLB.
 - It takes 100 nanoseconds to access memory for the page table and frame number.
 - It takes 100 nanoseconds to access the desired byte in memory.
 - A total of 220 nanoseconds.

Calculating Effective Access Time

$$\text{Effective access time} = 0.80 \times 120 + 0.20 \times 220 = 140 \text{ nanoseconds}$$

For a 98-percent hit ratio:

$$\text{Effective access time} = 0.98 \times 120 + 0.02 \times 220 = 122 \text{ nanoseconds}$$

General Formula

- Associative Lookup = time unit
- Can be less than 10% of memory access time
- Hit ratio = α (percentage of times that a page number is found in the associative registers)

$$\text{EAT} = (\alpha + \epsilon) + (2 + \epsilon)(1 - \alpha) = 2 + \epsilon$$

Memory Protection

Memory protection is implemented by associating protection bits with each frame. These bits are typically kept in the page table. Protection bits can indicate if read-only or read-write access is allowed.

Every memory reference goes through the page table to find the correct frame number. At the same time, the protection bits can be checked to verify that no writes are being made to a read-only page. An attempt to write to a read-only page causes a hardware trap to the operating system (or memory-protection violation).

More bits can be added to indicate page execute-only, and so on. A valid-invalid bit is attached to each entry in the page table:

- **Valid:** Indicates that the associated page is in the process's logical address space and is thus a legal page.
- **Invalid:** Indicates that the page is not in the process's logical address space.

Alternatively, a page-table length register (PTLR) can be used. Any violations result in a trap to the kernel.

Structure of the Page Table

Memory structures for paging can become very large. Consider a 32-bit logical address on modern computers:

- Logical Address Space = 2^{32}
- Page size of 4 KB (2^{12})

The page table would have 1 million entries ($2^{32}/2^{12}$). If each entry is 4 bytes, it would require 4 MB of physical address space/memory for the page table alone. This amount of memory used to cost a lot.

One solution is to divide the page table into smaller pieces:

1. **Hierarchical Paging**
2. **Hashed Page Tables**
3. **Inverted Page Tables**

Hierarchical Paging

Hierarchical paging involves breaking up the logical address space into multiple page tables. A simple technique is a two-level page table, where the page table itself is paged.

Admin $\Rightarrow$ Dept $\Rightarrow$ ClassNo $\Rightarrow$ Seat No in Class

Two-Level Paging Example

A logical address (on a 32-bit machine with 1K page size) is divided into:

- A page number consisting of 20 bits
- A page offset consisting of 12 bits

Since the page table is paged, the page number is further divided into:

- A 10-bit page number
- A 10-bit page offset

Thus, a logical address is as follows:

Misplaced \hline

Where:

- p_1 is an index into the outer page table
- p_2 is the displacement within the page of the outer page table

Because address translation works from the outer page table inward, this scheme is also known as a forward-mapped page table.

64-bit Logical Address Space

Even a two-level paging scheme may not be sufficient for a 64-bit logical address space.

- Logical Address Space = 2^{64}
- If page size is 4 KB (2^{12}), then the number of pages = $2^{64}/2^{12} = 2^{52}$

The page table has 2^{52} entries. In a two-level scheme:

- The outer page table has 2^{42} entries or 2^{44} bytes.

Three-level Paging Scheme

One solution is to add a 2nd outer page table. The outer page table is still 2^{32} bytes in size. The next step would be a four-level paging scheme, where the second-level outer page table itself is also paged, and so forth.

Hashed Page Tables

Hashed page tables are common in address spaces > 32 bits. The virtual page number is hashed into a page table. Each entry in the hash table contains a linked list of elements that hash to the same location (to handle collisions). Each element consists of three fields:

1. The virtual page number
2. The value of the mapped page frame
3. A pointer to the next element in the linked list

Algorithm

1. The virtual page number in the virtual address is hashed into the hash table.
2. The virtual page number is compared with field 1 in the first element in the linked list.
3. If there is a match, the corresponding page frame (field 2) is used to form the desired physical address.
4. If there is no match, subsequent entries in the linked list are searched for a matching virtual page number.

Inverted Page Table

Rather than each process having a page table and keeping track of all possible logical pages, inverted page tables track all physical pages. There is one entry for each real page (frame) of memory. The entry consists of the virtual address of the page stored in that real memory location, along with information about the process that owns that page. Thus, only one page table is in the system, and it has only one entry for each page (frame) of physical memory.

Architecture

Each virtual address in the system consists of a triple:

$\langle \text{process-id, page-number, offset} \rangle$

1. When a memory reference occurs, part of the virtual address, consisting of `<process-id, page-number>`, is presented to the memory subsystem.
2. The inverted page table is then searched for a match.
3. If a match is found—say, at entry i —then the physical address `<i, offset>` is generated.
4. If no match is found, then an illegal address access has been attempted.

Segmentation

Segmentation is a memory-management scheme that supports the user's view of memory. A program is a collection of segments, where each segment is a logical unit such as:

- Main program
- Procedure
- Function
- Method
- Object
- Local variables
- Global variables
- Common block
- Stack
- Symbol table
- Arrays

User's View of a Program

A logical address space is a collection of segments. Each segment has a name and a length. Addresses specify both the segment name and the offset within the segment.

Misplaced \hline

Segmentation Architecture

Segments are numbered and are referred to by a segment number rather than by a segment name. A logical address consists of a two-tuple:

`<segment-number, offset>`

The compiler automatically constructs segments reflecting the input program. A C compiler might create separate segments for:

1. The code
2. Global variables
3. The heap, from which memory is allocated
4. The stacks used by each thread
5. The standard C library

Segmentation Hardware

Segmentation hardware maps two-dimensional physical addresses. Each table entry has:

- **Base:** Contains the starting physical address where the segments reside in memory.
- **Limit:** Specifies the length of the segment.

The segment number is used as an index to the segment table. The offset d of the logical address must be between 0 and the segment limit. If it is not, the system traps to the operating system (logical addressing attempt beyond the end of the segment).

When an offset is legal, it is added to the segment base to produce the address in physical memory of the desired byte. The segment table is thus essentially an array of base-limit register pairs.

- **Segment-table base register (STBR):** Points to the segment table's location in memory.
- **Segment-table length register (STLR):** Indicates the number of segments used by a program; segment number s is legal if $s < \text{STLR}$.

Segmentation Example

Consider five segments numbered from 0 through 4. The segments are stored in physical memory. The segment table has a separate entry for each segment, giving the beginning address of the segment in physical memory (base) and the length of that segment (limit).

For example:

- Segment 2 is 400 bytes long and begins at location 4300. A reference to byte 53 of segment 2 is mapped onto location $4300 + 53 = 4353$.
- A reference to segment 3, byte 852, is mapped to 3200 (the base of segment 3) + 852 = 4052.
- A reference to byte 1222 of segment 0 would result in a trap to the operating system, as this segment is only 1000 bytes long.

Example: The Intel 32 and 64-bit Architectures

Intel IA-32 Architecture (Segmentation with Paging)

- Supports both segmentation and segmentation with paging.
- Each segment can be 4 GB.
- Up to 16K segments per process.
- Divided into two partitions:
 - First partition of up to 8K segments is private to the process (kept in the local descriptor table (LDT)).
 - Second partition of up to 8K segments is shared among all processes (kept in the global descriptor table (GDT)).

Logical to Physical Address Translation in IA-32

1. The CPU generates logical addresses, which are given to the segmentation unit.
 2. The segmentation unit produces a linear address for each logical address.
 3. The linear address is then given to the paging unit, which in turn generates the physical address in main memory.
 4. Thus, the segmentation and paging units form the equivalent of the memory-management unit (MMU).
- CPU generates logical addresses.
 - Selector given to segmentation unit, which produces linear addresses.
 - Linear address given to paging unit, which generates physical address in main memory.
 - Paging units form the equivalent of MMU.
 - Pages sizes can be 4 KB or 4 MB.

Intel IA-32 Page Address Extensions

32-bit address limits led Intel to create page address extensions (PAE), allowing 32-bit apps access to more than 4GB of memory space. Paging went to a 3-level scheme. The top two bits refer to a page directory pointer table. Page-directory and page-table entries moved to 64-bits in size. The net effect is increasing address space to 36 bits (64GB of physical memory).

Intel x86-64

- Current generation Intel x86 architecture.
- 64 bits is ginormous (> 16 exabytes).
- In practice, only implement 48-bit addressing.
- Page sizes of 4 KB, 2 MB, 1 GB.
- Four levels of paging hierarchy.
- Can also use PAE so virtual addresses are 48 bits and physical addresses are 52 bits.

Shared Pages

Shared Code

One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems). Similar to multiple threads sharing the same process space. Also useful for inter-process communication if sharing of read-write pages is allowed.

Private Code and Data

Each process keeps a separate copy of the code and data. The pages for the private code and data can appear anywhere in the logical address space.